

7. PROJECT DOCUMENTATION PHASE

7.1 Documentation Structure and Technical Description

Project documentation explains the problem, objectives, scope, requirements, design, implementation, testing and limitations. The introduction describes the need for structured cricket records. Requirements define features and quality expectations. Design explains architecture, database relationships, workflows and interface decisions. Implementation describes modules and calculation rules. Testing presents test cases and results.

Technical descriptions must match the actual application. If the project uses SQLite locally, the document should not claim a cloud database. If scorecards are entered manually, it should not describe a live scoring integration. Diagrams should use the same entity names and relationships as the implemented schema. Screenshots should come from the final version and include informative captions.

Domain terms such as innings, dismissal, balls faced, legal delivery, economy rate and strike rate should be defined. The document should explain how not-outs, extras and unavailable metrics are handled. This prevents users from interpreting a blank value, zero or N/A as equivalent.

The documentation should be organized with consistent headings, figure numbers, table captions and page numbering. References should be included for external libraries, frameworks and statistical definitions. The document should distinguish implemented features from proposed future work so that readers understand the actual system capabilities.

7.2 User Manual and Operating Instructions

The user manual begins with system requirements and installation instructions. It should explain how to create the environment, install dependencies, initialize the database and launch the application. Any default credentials or setup configuration should be documented securely and changed before real deployment.

Instructions should be task-based. For player registration, identify required fields, explain how to save and show how to verify the profile appears in the player list. For match recording,

explain team selection, match details, innings entry, performance fields, validation messages and saving. For reports, explain how filters work and define the displayed metrics.

Troubleshooting should cover common problems such as missing dependencies, database errors, invalid numeric input and empty reports caused by restrictive filters. The manual should describe where exported files are saved and how records can be backed up and restored. Screenshots may support complex steps but must be legible and reflect the current interface.

A quick-start section can help new users complete the core workflow: register a player, create a match, enter performance data and view a report. Clear operating instructions reduce training effort and help users distinguish application errors from incorrect source scorecard information.

7.3 Maintenance, Backup and Limitations

Maintenance documentation should identify the application version, programming language, library versions, database schema and configuration files. It should explain how to update dependencies, back up the database, restore a backup and run tests after changes. Schema updates should use a repeatable migration or initialization process, with a backup taken before modifying important records.

Known limitations should be stated accurately. Manual entry depends on scorer accuracy. Metrics may be affected by differences in match formats and scoring conventions. Small datasets may not support meaningful long-term comparisons. The application does not predict future player performance unless a separate validated model is implemented. Statistics should not be treated as the sole basis for selection or training decisions.

Future enhancements may include ball-by-ball scoring, tournament management, role-based access, secure multi-device synchronization, richer exports, standard scorecard import and configurable metric definitions. Each enhancement should be assessed for user value, data quality, privacy and development effort.

A maintenance plan should identify who is responsible for reviewing defects and updating dependencies. Backups should be stored securely and restoration should be tested periodically if the system is used for ongoing records. Documenting limitations and maintenance responsibilities helps prevent users from assuming unsupported capabilities.

7.4 References, Appendices and Final Review

The final document should cite programming frameworks, libraries, database technologies and any external cricket-statistics definitions used. References should follow the institution's required citation style. Technical descriptions should be paraphrased appropriately, and copied material should not be presented as original work. If sample scorecards or datasets are used, their source should be identified or they should be described as synthetic.

Appendices may include a requirements traceability matrix, database schema, selected code excerpts, test cases, sample reports, installation instructions and user feedback forms. Appendices should support understanding and verification rather than reproduce large amounts of irrelevant source code. Passwords and private player information must not be included.

Before submission, review page numbering, headings, table and figure captions, spelling, references and cross-references. Confirm that every diagram matches the implemented system and that screenshots do not contain temporary test errors or confidential information. Ask another person to follow the installation guide and user manual to identify unclear steps.

The final review should confirm that the documentation accurately describes the delivered application. Completed features, known defects, environmental assumptions and future enhancements should be clearly separated. This creates a reliable record for evaluation, handover and future development.